

Deep Active Learning for Text Classification

Bang An
BeiHang University
Beijing, China
No.37 XueYuan Road
(0086)18612210095
anbang@buaa.edu.cn

Wenjun Wu
BeiHang University
Beijing, China
No.37 XueYuan Road
(0086)13641084384
wwj@nlsde.buaa.edu.cn

Huimin Han
BeiHang University
Beijing, China
No.37 XueYuan Road
(0086)13051087129
hanhuimin@nlsde.buaa.edu.cn

ABSTRACT

In recent years, Active Learning (AL) has been applied in the domain of text classification successfully. However, traditional methods need researchers to pay attention to feature extraction of datasets and different features will influence the final accuracy seriously. In this paper, we propose a new method that uses Recurrent Neural Network (RNN) as the acquisition function in Active Learning called Deep Active Learning (DAL). For DAL, there is no need to consider how to extract features because RNN can use its internal state to process sequences of inputs. We have proved that DAL can achieve the accuracy that cannot be reached by traditional Active Learning methods when dealing with text classification. What's more, DAL can decrease the need of the great number of labeled instances for Deep Learning (DL).

At the same time, we design a strategy to distribute label work to different workers. We have proved by using a proper batch size of instance, we can save much time but not decrease the model's accuracy. Based on this, we provide batch of instances for different workers and the size of batch is determined by worker's ability and scale of dataset, meanwhile, it can be updated with the performance of the workers.

CCS Concepts

• Computing methodologies→Machine learning→Learning settings→Active learning settings.

Keywords

Active Learning; Deep Learning; Machine Learning; Artificial Intelligence; Text Classification.

1. INTRODUCTION

In the past few years, Machine Learning has made great progress in a lot of application areas. However, as time goes by, there have been many new fields and new issues for researchers to explore. In many real-world applications, the labels are difficult to obtain because of the scarce of experts or the lack of money. In order to deal with these issues, Active Learning is proposed.

Active Learning is a setup that allows the learning algorithm to iteratively and strategically query the labels of some instances for reducing human labeling efforts.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ICVIP 2018, August 27–29, 2018, Las Vegas, NV, USA
© 2018 Association for Computing Machinery.
Copyright 2018 ACM ISBN 978-1-4503-6529-1/18/08 \$15.00
DOI: <https://doi.org/10.1145/3271553.3271578>

The model is trained on a small number of datasets firstly, and an acquisition function (often based on the model's uncertainty) decides which instance to select. After an oracle (often a human expert) labels the selected instance, it will be added to the training dataset, and a new model will be trained on the updated training dataset. The process will be repeated again and again with the training dataset increasing in size over time [3].

Text classification is a classic topic for Natural Language Processing (NLP) and has many applications in topics such as parsing, semantic analysis, information extraction and web searching [1]. Nowadays the core task in text classification is how to present features. In spite of the fact that many researchers have developed some more complex methods in order to extract more contextual information and accurate word order, but there still exist few issues such as data sparseness or data insufficiency which impact the classification accuracy deeply.

It's well known that the key idea lying behind active learning is a machine learning algorithm can achieve greater accuracy with fewer training labels if it is allowed to choose data from which it learns [13]. Although traditional AL has been applied in the domain of text classification successfully, also by definition fails to address some important questions:

First, it doesn't solve with the feature extraction issue in text processing. It means researchers still need to compare with different methods in order to select which one is best, because the choice of method will impact the result of the model training seriously.

Second, it always costs too much time to finish the task and is not effective to assign label works to many workers. The size of train dataset will be bigger and bigger as the acquisition function selects instance one by one, and a bigger train dataset means it would take more time for the model to train one time. Meanwhile the labeling task cannot be assigned to many workers at the same time efficiently. On the one hand, the workers have to wait until the acquisition function selects the instance, on the other hand, it will be a big challenge for the model to response if too many workers label at the same time.

In recent years, the rapid development of word embedding and deep neural networks has made a great progress to various NLP tasks especially for text classification. RNN can analyze a text word by word and store the semantics of all the previous text in hidden layer, with this advantage it can capture the contextual information easily. Which means it may have the ability to achieve higher accuracy in classification than traditional method. What's more, with the help of word embedding, some composition-based methods are proposed to capture the semantic representation of texts [9]. By this way, researchers don't need to spend time on feature extraction any more. However, it is obvious that deep neural networks depend on a big number of labeled instances and too much computational resource.

Based on these issues, we consider to combine Deep Learning with Active Learning which we call it Deep Active Learning. We select RNN because it has the advantage of capturing semantics of variable-length texts. Based on the advantage of RNN, we don't need to spend time on comparing different algorithms of feature extraction and can achieve lower error rates. With the advantages of Active Learning, we can save around 50% labor to get the labeled instances [17]. What's more, we improve the efficiency of traditional Active Learning, we select batch-mode for Active Learning and design a new strategy that can distribute label work to different workers.

This paper is organized as follows: section 2 introduces the related work for traditional Active Learning, text classification and deep learning. In section 3 we propose two models, one is traditional active learning model for text classification and the other is DAL model. In section 4 we design two parts of experiments, one for demonstrating our DAL can achieve higher accuracy and the other for demonstrating that our distribute strategy can not only save time but also maintain the performance of traditional AL methods. Finally, we conclude in section 5 with some discussion.

2. Related Work

2.1 Active Learning Strategies

There are two setups of active learning for multiclass classification: stream-based and pool-based. For stream-based setup, the instances come in sequence and the algorithm has to decide whether to query the label of the instance or discard it. By contrast, the pool-setup is more flexible when we try to deal with practical problems, because in real world we often gather all unlabeled data at once instead of one by one. There are two pools for pool-based setup, one called the labeled pool and the other called the unlabeled pool. We treat the whole data pool including n instances by using $D = \{x_1, x_2, \dots, x_{n_u}, (x_{n_u+1}, y_{n_u+1}), \dots, (x_n, y_n)\}$, where the input instance $x_i \in R^d$ and y_i is the label of x_i . The D is the combination of the unlabeled pool D_u which contains the first n_u instances, and the labeled pool D_l which contains other instances. The algorithm will select the most informative instance from the D_u to query its label, then the instance will be moved to the D_l , next the model will be trained with the updated D_l and finally select the new instance circularly. In the widely used pool-based approach, we start with a small labeled training set D_l and a large pool of unlabeled data D_u [7].

The choice of instance to be labeled is done through an acquisition function, which ranks points based on their potential informativeness. There are some frameworks for measuring informativeness such as uncertainty sampling and query-by-committee (QBC).

The main idea of uncertainty sampling is to select the instance which is the least certain to label. Probabilistic model will be used in this algorithm, for example, a more general uncertainty sampling variant might query the instance whose prediction is the least confident as equation (1):

$$x_{LC}^* = \operatorname{argmax}_x 1 - P_{\theta}(\hat{y}|x) \quad (1)$$

where $\hat{y} = \operatorname{argmax}_y P_{\theta}(y|x)$, or the class label with the highest posterior probability under the model θ [13].

Another general active learning framework is QBC. We use a committee of models $\mathcal{C} = \{\theta^{(1)}, \dots, \theta^{(C)}\}$ to represent C different hypotheses that are consistent with D_l . The most

informative query is the instance over which the committee is in most disagreement about how to label.

Vote Entropy (VE) is the measure to select the instance in QBC and the formula is as equation (2):

$$x_{VE}^* = -\frac{1}{T} \sum_{t=1}^T \sum_{m=1}^M \frac{V(y_t, m)}{c} \log \frac{V(y_t, m)}{c} \quad (2)$$

where $V(y_t, m)$ is the number of "votes" label m receives from all the committee member's label work at sequence position t [14].

Compared with these two popular frameworks, although in some scenarios the QBC is proved more effective than uncertainty sampling, we have to commit the time it needs is a few times longer because it will go through many "votes" from multiple committee members. As we know Deep Learning is time consuming and dependent on computing resources, so it is not easy to combine with QBC.

2.2 Text Classification

Text classification is very important nowadays with the recent explosion of available text data on the Internet, it mainly focus on three topics: feature engineering, feature selection and using different types of machine learning algorithms. For feature engineering, the most widely used feature is the bag-of-words. In addition, some more complex features have been designed such as part-of-speech tags, noun phrases and tree kernels [9]. In the bag-of-words model, a text is often represented as the bag of its words, which means disregard sentence grammar and even word order but only keep multiplicity.

Term Frequency Inverse Document Frequency (TF-IDF) is a numerical statistic that is intended to reflect how important a word is to a document in a collection or corpus [11]. Higher TF-IDF values words means that have imply they have a stronger relationship in the document which they appear. And it has been proved that the combination of bag-of-words and TF-IDF can outperform either bag-of-words or TF-IDF [10].

After feature extraction the machine learning algorithms often use classifiers such as logistic regression (LR), naive bayes (NB) and support vector machines (SVM). Compared with logistic regression, SVM can analyze data used for multiclass classification, linear classification and non-linear classification using what is called the kernel trick. Besides, different experimental results have proved that SVM performs better than NB in general classification tasks, and average improvement is +6.36% or even +28.78% by using SVM [5].

Although SVM is useful in many cases of classification, it is very sensitive to outliers and noises in the training dataset, meanwhile, cannot realize the feature selection. And Deep Learning can solve with these issues.

2.3 Deep Learning for Text Classification

Deep Learning is part of a broader family of machine learning methods based on learning data representations. And the famous architectures such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) have been successfully applied to fields including speech recognition, computer vision, text classification and so on.

CNN has made a great success in the domain of text classification, it has been trained with one layer of convolution on top of word vectors obtained from a neural language model [8]. Compared with CNN, the advantage of RNN is the ability to better capture the contextual information. This could be beneficial to capture

semantics of variable-length texts [9]. Gating mechanisms have been developed for basic RNN, resulting in two prevailing types one called long short-term memory (LSTM) and the other called gated recurrent unit (GRU).

It has been proved RNNs perform well and robust in a broad range of tasks except when the task is essentially a key-phrase recognition task as in some sentiment detection and question-answer matching settings [20]. But current literature does not support such a clear conclusion which is better between LSTM and GRU.

There is no doubt that Deep Learning depends on a larger number of dataset than traditional method. What's worse, obtaining more labeled instances is expensive and even unrealistic in practical applications. So we consider to take advantage of the concept of Active Learning. By this way, the RNN will select most instances to train every batch and just need a smaller number of labeled instances to achieve final accuracy than traditional ways.

2.4 Batch-Mode for Active Learning

In traditional Active Learning, only one instance will be selected at a time which lead train speed of model is very slow. By contrast, batch-mode active learning allows the learner to query instances in groups, which is better suited to parallel labeling environments or models with slow training procedures [13]. It is more feasible because it does not require model to be re-trained after an oracle finish one selection, by this way the batch-mode can make good use of human labor.

There are some approaches for batch-mode. And a simple strategy is to select the top k most informative examples called batch-greedy. Which means the approach will greedily select examples with a batch, and assemble batches in a greedy manner. Other methods such like using the Fisher information matrix for the measurement of model uncertainty [6] or optimal batch-model policy [2] have been proved using batches only incurs a bounded increase of cost as compared to allowing fully sequential selection and more effective. However, all the algorithms only focus on how to select instances or which instance to select. They fail to answer how many instances to select one batch.

3. Proposed Model

3.1 Traditional Active Learning

3.1.1 Feature Extraction

To deal with the raw input from the train dataset, we use the method which combines of bag-of-words and TF-IDF. First we select the top 3,000 words that shows most frequent in the bag-of-words, and then calculate TF-IDF.

As the term implies, TF-IDF calculates values for each word in a document through an inverse proportion of the frequency of the word in a particular document to the percentage of documents the word appears in [12], so it is very useful in text mining. There are some different methods for TF-IDF such as boolean frequencies, term frequency adjusted for document length and raw count, and the formula we select is as equation (3):

$$tfidf(t,d,D) = tf(t,d) * idf(t,D) = \frac{f_{t,d}}{\sum_{t \in d} f_{t,d}} * \log \frac{N}{|\{d \in D : t \in d\}|} \quad (3)$$

where N is the total number of documents in the corpus $N = |D|$, and $|\{d \in D : t \in d\}|$ number of documents where the term t appears. After dealing with the dataset we select SVM as the classifier.

3.1.2 Support Vector Machines as The Classifier

For text classification, we will always receive the training dataset $\{x_1, x_2, x_3, \dots, x_n\}$ and their labels $\{y_1, y_2, y_3, \dots, y_n\}$ at the same time. If all the training data are vectors in some space $\mathcal{X} \subseteq \mathbb{R}^d$, then all the training instances lie close to the hyperplane called support vectors. And more generally, SVM allow one to project original training instance in space $\mathcal{X}$ to a higher dimensional feature space $\mathcal{F}$ via a kernel operator K . In a word, the set of classifiers is equation (4) [18]:

$$f(x) = (\sum_{i=1}^n a_i K(x_i, x)) \quad (4)$$

The kernel we select is Radial Basis Function (RBF), it uses the equation (5) to represent the feature vectors in input space for two samples x and x' :

$$K(x, x') = \exp(-\frac{\|x - x'\|^2}{2\sigma^2}) \quad (5)$$

3.1.3 Measures to Select Instances

Because we aim to deal with multiclass classification, the least confidence method has a noticeable disadvantage. Imagine there are three classes, the probability that instance a belongs to three classes is 0.5, 0.5, 0 and instance b is 0.5, 0.25, 0.25. The least confidence will give the same value for instance a and instance b , however, it's obvious that instance a is more uncertain than instance b . Which means the instance with highest score may not be the most informative one.

In order to address this problem, we select to use the method called margin sampling, as equation (6) show:

$$x_{MS}^* = P_\theta(\hat{y}|x) - P_\theta(\hat{\bar{y}}|x) \quad (6)$$

where $\hat{\bar{y}}$ is the second most probable class. Every turn we calculate informativeness among all instances and use the batch-greedy method to select instances in batch.

3.2 Deep Active Learning

Based on these related works we mentioned above, we propose a new model which uses RNN as the acquisition function in AL, and the structure is shown in figure 1:

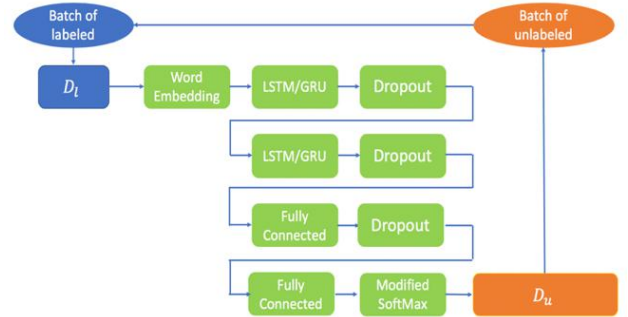


Figure 1. The structure of DAL.

3.2.1 Word Embedding to Deal With The Input Data

To deal with the input data, we use the word embedding and set the embedding size 800 in all cases. When given a sentence fragment $F = \{w_1, w_2, \dots, w_T\}$, where T is the sentence length, we need to get its one-hot vector representation v_t first and then transform each word in fragment in to a real-valued vector e_t via looking up the embedding matrix $W^{word} \in \mathbb{R}^{d_w \times V}$. That is equation (7):

$$e_t = W^{word} v_t \quad (7)$$

where d_w is the size of word embeddings and V is the size of vocabulary. Moreover, the embedding matrix W^{word} needs to be learned. With the help of word embedding, some composition-based methods are proposed to capture the semantic representation of texts.

3.2.2 The structure of Deep Neural Networks

RNN are sequence-based models of key importance for natural language understanding, language generation and many other tasks. The input of the model is seen as a sequence of symbols, where at each time step a simple neural network is applied to a single symbol, as well as to the network's output from the previous time step.

By this way, we don't need to consider how to extract features from the input data. For example, given input sequence $x = [x_1, x_2, \dots, x_n]$ of length N , a simple RNN is formed by a repeated application of a deterministic function f_h . This generates a hidden state h_t for time step t [4], as equation (8) show:

$$h_t = f_h(x_t, h_{t-1}) = \sigma(x_t W_h + h_{t-1} U_h + b_h) \quad (8)$$

where W is the weight matrix. For some non-linearity σ . The model output can be defined such as equation (9):

$$f_y(h_T) = h_T W_y + b_y \quad (9)$$

As we mentioned in section 2.3, there is no such a clear conclusion which is better between LSTM and GRU. So we set two models for DAL, one core is LSTM and the other is GRU.

LSTM use different gates within the RNN units. The LSTM maintains a separate memory cell inside that updates and exposes its content only when necessary. The input at time t is x_t, h_{t-1}, c_{t-1} , and output is h_t, c_t , all of them can be updated by equations (10-15) [19]:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (10)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \quad (11)$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (12)$$

$$g_t = \tanh(W_g x_t + U_g h_{t-1} + b_g) \quad (13)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot g_t \quad (14)$$

$$h_t = o_t \odot \tanh(c_t) \quad (15)$$

At the same time GRU models the word sequence x as follow equations (16-19):

$$z = \sigma(x_t U_z + h_{t-1} W_z) \quad (16)$$

$$r = \sigma(x_t U_r + h_{t-1} W_r) \quad (17)$$

$$s_t = \tanh(x_t U_s + (h_{t-1} \odot r) W_s) \quad (18)$$

$$h_t = (1 - z) \odot s_t + z \odot h_{t-1} \quad (19)$$

where σ denotes the logistic sigmoid function and $\odot$ denotes the element-wise vector product. And $U \in \mathbb{R}^{d \times h}$, $W \in \mathbb{R}^{h \times h}$.

Next, we add a dropout layer after each RNN layer so as to avoid over-fitting problem. Through many experiments the value of the probability of retaining a unit p we select is 0.8 [15].

Finally, there are two fully connected layers in our structure to deal with the high-level reasoning. The first one has connections to the output of RNN layer. Connected with the first fully connected layer is another dropout layer and relu layer. After relu layer applies the non-saturating activation function $f(x) =$

$\max(0, x)$, the second fully connected layer connects with the modified Softmax layer.

3.2.3 Method to Select Instances

As we mentioned above, uncertainty sampling is a very straightforward probabilistic learning model. So we decide to combine Softmax with the uncertainty sampling (equation 20) to select the instances with most informativeness based on what we mentioned in section 2.1 and section 3.1.3:

$$[p_\theta(y = 1|x), \dots, p_\theta(y = C|x, \theta)] = \text{Softmax}(f^\theta(x)) \quad (20)$$

Through this function we will get the probability that an instance belongs to each class. The least confident one is the instance whose probability in each category is closest. The calculate formula is as equation (21):

$$x_{\text{Softmax}}^* = P_\theta(\hat{y}|x) - P_\theta(\bar{y}|x) \quad (21)$$

where $\hat{y}$ belongs to $\{1, 2, \dots, C\}$ as equation (22) show,

$$P_\theta(\bar{y}|x) = \frac{\text{Softmax}(f^\theta(x))}{C} \quad (22)$$

By this way, we can get the most informative instances in batch and then add them into the labeled pool, the RNN model will be re-trained with the updated train dataset iteratively.

4. Experiment

We conducted two parts of experiments to validate the proposed framework. For the first experiment we planned to prove that DAL can achieve the higher accuracy with fewer labeled instances. We selected the public datasets named THUCNews which contains 10 class and 10000 instances [16]. We compared LSTM core and GRU core in the beginning and selected the better one to join the comparative experiments.

In the second experiment we planned to validate that by using the proper batch size for AL we can save much time but not decrease the accuracy. And base on the experiment we want to find relation between time-consuming and batch number in order to design the distribute strategy. We also used the same dataset but decreased the number of data to 3000 because the size of data was not key factor.

4.1 The Experiment of DAL

We designed three sets of experiments:

For Traditional AL, we used bag-of-words and TF-IDF to deal with the dataset, we selected SVM as the classifier and uncertainty sampling with margin sampling. The structure was shown in section 3.1.

For Random, we used bag-of-words and TF-IDF to deal with the dataset, we selected SVM as the classifier but selected the instances randomly.

For DAL, we didn't need to extract features as we mentioned above, so we just used word embedding and finally modified the uncertainty sampling with Softmax. The structure was shown in section 3.2.

As we mentioned in section 2.1, both the DAL and Traditional AL first selected the same labeled training set D_l to pre-train. And then selected the most informative instances in the same batch size and the size is 64 from D_u iteratively. The procedure of experiments is as figure 2:

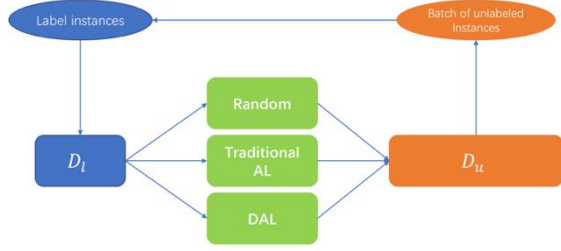


Figure 2. The procedure of experiments.

First of all, just as what we said in section 2.3, we need to compare which one is better between LSTM core and GRU core in DAL, all experiment conditions are the same except the RNN core. The result is as table 1:

Table 1. Comparison between LSTM and GRU.

	LSTM core	GRU core
F1 value	85.6	90.3
Time-consuming	19h 57mins	21h 32mins

As we can see from table 1, although GRU takes more than 1 hour than LSTM, GRU behaves better with the dataset in DAL. So we choose GRU as the RNN core and start to do the comparison experiment finally.

And the result of experiments is as figure 3:

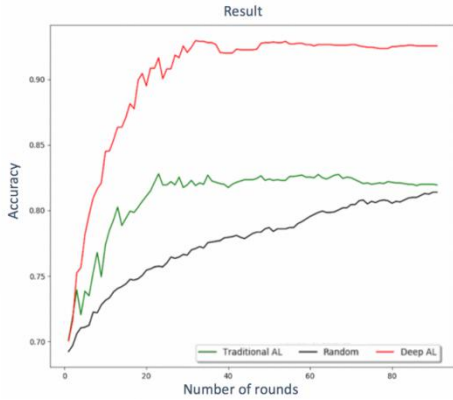


Figure 3. The result of the first experiment.

From figure 3 we can see the model can not only achieve higher accuracy after each label turn, but also can achieve the final accuracy that traditional models cannot reach. What's more, it's noticeable that after 40 rounds both the traditional AL and the DAL have reached a high stable accuracy, which means DAL can reach final accuracy by using only 45% of the initial dataset with the advantage of Active Learning.

However, we have to admit the DAL will spend three times as much time as traditional AL, besides it depends on large number of data to pre-train. Time-consuming of three sets of experiments is shown in table 2.

Table 2 Time-consuming of the first experiment

	TAL	Random	DAL
Time-consuming	9h 9mins	3h 10mins	21h 55mins

4.2 The Experiment of Different Batch Size

First we wanted to prove that using different batch size would save time but not decrease the model's accuracy so we designed three sets of experiments:

The first one we selected batch size for only one instance, the second one 16 instances and the third one 64 instances. And because the first experiment would cost too much time, we decreased the size of dataset to 3,000 instances.

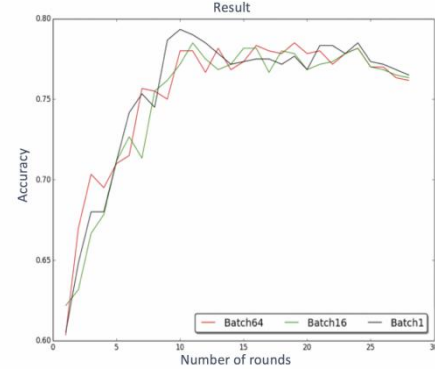


Figure 4. The result of the second experiment.

As we can see from the figure 4, there is no significant difference in accuracy among the three experiments. However, the difference in time-consuming is very obvious which is shown in the table 3:

Table 3. Time-consuming of the second experiment.

	Batch 1	Batch 16	Batch 64
Time-consuming	18h 50mins	1h 14mins	19mins

It's obvious that it will save 99% time by using batch size 64 instead of only one instance. Although there is no significant difference among accuracy results, it doesn't mean the bigger the batch size, the better. On the one hand, it will not reflect the advantages of active learning if the size is too big (consider to return all unlabeled instances in one turn). On the other hand, the label job of batch size needs to fit the label worker's ability in real application. In order to discuss the relation between time-consuming and batch size, we also record the time-consuming in each turn which is shown as figure 5:

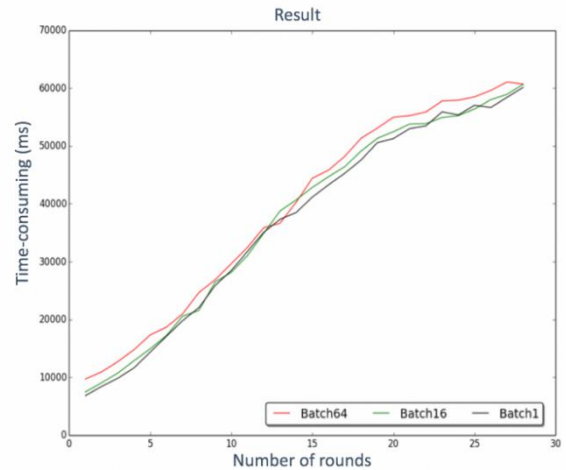


Figure 5. Time-consuming for each turn.

As we can see from the figure 5, time-consuming is getting bigger and bigger with the increase of the labeled dataset in each turn and there is no apparent difference among the three sets of batch size. Meanwhile, we recognize if we maintain the batch size in a constant then workers have to wait longer and longer as they finish more label work. So we design a strategy to distribute batch of instances to different workers. Assume m is the number of workers and all the workers have the same ability a . W is the size of all unlabeled instances initially, the equation of work load for every worker should be equation (23):

$$\frac{W}{m} = a * (\sum_{i=1}^{n-1} k + n * t) \quad (23)$$

where t is time-consuming that a worker needs to spend on finishing initial batch size, k is coefficient of time increasing with the label work (can be seen as a constant approximately in Figure 5) and n is total batch number. We can calculate n easily, so finally the size of batch should be equation (24):

$$S = (x * k + t) * a \quad (24)$$

where x in $\{1, 2, \dots, n\}$. In ideal condition the total time we save should be equation (25):

$$T = \left(\frac{(k-2t+\sqrt{\Delta})^2 + 4t-2k-2\sqrt{\Delta}}{8} \right) * k * m \quad (25)$$

where $\Delta = \frac{8W+ma(2t-k)^2}{ma}$. By this way, we can save much time but not decrease accuracy of the model.

5. Conclusion

We propose a novel framework for active learning which we call it Deep Active Learning. In DAL we don't need to consider how to extract features because RNN can capture the contextual information, meanwhile it can achieve a higher accuracy than traditional AL when dealing with the text classification problem. What's more DAL can reach the high stable precision by only using 45% of the initial dataset. But we have to commit DAL will consume much more time and need more labeled instances to pre-train than traditional AL method.

Refer to DAL, we also modify the traditional framework of AL, we design a strategy that assign label work to different workers. We select the batch size of instances from the unlabeled pool and the batch size is determined by the worker's ability and the scale of unlabeled pool. What's more it can be updated as the performance of workers. By this way we can save much time but not decrease the model's accuracy.

REFERENCES

- [1] Aggarwal, C. C., and Zhai, C. 2012. A survey of text classification algorithms. In *Mining text data*. Springer. 163-222.
- [2] Chen, Y. and Krause, A. 2013. Near-optimal batch mode active learning and adaptive submodular optimization. In *ICML'13 Proceedings of the 30th International Conference on International Conference on Machine Learning*.
- [3] Gal, Y., Islam, R. and Ghahramani, Z. 2016. Deep Bayesian Active Learning with Image Data. In: *Advances in Neural Information Processing Systems* 29.
- [4] Gal, Y. 2016. Uncertainty in Deep Learning. Doctor. University of Cambridge.
- [5] Hassan, S., Rafi, M. and Shaikh, M. 2011. Comparing SVM and naïve Bayes classifiers for text categorization with Wikitology as knowledge enrichment. In *IEEE 14th International Multitopic Conference*.
- [6] Hoi, S., Jin R., Zhu J. and Lyu M. 2006. Batch Mode Active Learning and Its Application to Medical Image Classification. In *ICML '06 Proceedings of the 23rd international conference on Machine learning*.
- [7] Huang, K. and Lin, H. 2016. A Novel Uncertainty Sampling Algorithm for Cost-sensitive Multiclass Active Learning. In *2016 IEEE 16th International Conference on Data Mining (ICDM)*.
- [8] Kim, Y. 2014. Convolutional Neural Networks for Sentence Classification. In *Empirical Methods in Natural Language Processing*.
- [9] Lai, S., Xu, L., Liu, K. and Zhao, J. 2015. Recurrent Convolutional Neural Networks for Text Classification. In *Twenty-Ninth AAAI Conference on Artificial Intelligence*
- [10] Lilleberg, J., Zhu, Y. and Zhang, Y. 2015. Support Vector Machines and Word2vec for Text Classification with Semantic Features. In *IEEE 14th International Conference on Cognitive Informatics & Cognitive Computing*.
- [11] Rajaraman, A., Ullman, J.D. 2011. "Data Mining". *Mining of Massive Datasets*. pp. 1-17. DOI = 10.1017/CBO9781139058452.002. ISBN 978-1-139-05845-2.
- [12] Ramos, J.E. 2003. Using TF-IDF to Determine Word Relevance in Document Queries.
- [13] Settles, B. 2010. Active Learning Literature Survey.
- [14] Settles, B. and Craven, M. 2008. An Analysis of Active Learning Strategies for Sequence Labeling Tasks. In *Empirical Methods in Natural Language Processing*.
- [15] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15(1929-1958).
- [16] Sum, M., Li, J., Guo, Z., Zhao, Y., Zheng, Y., Si, X. and Liu, Z. 2016. THUUCTC: An Efficient Chinese Text Classifier.
- [17] Thompson, C., Califf, M. and Mooney, R. 1999. Active Learning for Natural Language Parsing and Information Extraction. In *Proceedings of the Sixteenth International Machine Learning Conference*.
- [18] Tong, S. and Koller, D. 2001. Support Vector Machine Active Learning with Applications to Text Classification. *Machine Learning Research*.
- [19] Wang, X., Jiang, W. and Luo, Z. 2016. Combination of Convolutional and Recurrent Neural Network for Sentiment Analysis of Short Texts. *Osaka, Japan*, pp.2428-2437.
- [20] Yin, W., Kann, K., Yu, M. and Schütze, H. 2017. Comparative Study of CNN and RNN for Natural Language Processing.